



UNDERSTANDING CONSTRAINTS: A FRAMEWORK FOR TECH ART DECISION MAKING

Alexander Zotikov, Technical Artist

“Decision making in Technical Art”, “Decision Making”.. sounds like a title for a thick book or DVD with a training course. Relax, I’m not going to give you detailed instructions on how to decide what to do in a given situation, or even worse, pretend, that I know how to do it myself.

In the next 25 minutes I’m going to tell you the story about Remedy Environment team and how we decided to changed a big part of our pipeline, why was it possible, how hard was it and what findings we collected during this process. You’ll hear a bit about uncertain technical artists and brave environment artists and far-sighted tools programmers. I’ll supplement the story with some technical solutions as well.

INTRODUCTION

- 7 years in the industry
- 6 as a Technical Artist
- Previously worked on:



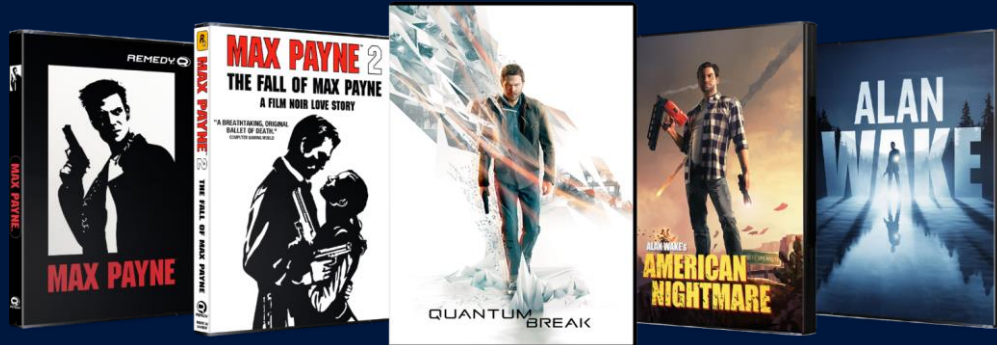
My name is Alexander Zotikov. I've been working in the game industry for about 7 years. Started my career as a 3D Artist, but quickly switched to the Technical Art role.

I was responsible for different areas starting from exporting and setting up content in the game engine, making destruction to developing tools for the team of 50+ artists.

As a Technical Artist I've shipped World of Warships at Wargaming, and then was contributing to the Rainbow Six: Siege.

REMEDY ENTERTAINMENT

BEST KNOWN FOR:



- Founded in 1995
- Based in Espoo, Finland
- 200+ employees



One and a half years ago I've joined Remedy Entertainment team. Remedy is an independent game studio based in Espoo, Finland. You may know us for the games like Max Payne, Alan Wake and Quantum Break.

REMEDY ENTERTAINMENT

CURRENTLY WORKING ON:



n^orthlight[®]
REMEDY STORYTELLING TECHNOLOGY



At the moment, we are working on the game called Control (supernatural third-person action-adventure) and also, we are making the story mode for the CrossFire 2 (FPS).

Need to mention, that both games are based on our own game technology called Northlight.

Speaking about technologies and game development, we all use pre-production to plan ahead, build the systems that will solve our future issues.

IMMEDIATE SOLUTIONS VS. SYSTEM



System with immediate solutions

REMEDY 

In the middle of production or later you need immediate solutions, that works “now”.
If you support tools for multiple projects which overlap, you are often in “production”/”shipping” mode, so you may end up creating a lot of temporary solutions or “patches”.
Some of those solutions migrate from project to project.
It doesn’t necessary mean that immediate solutions are bad. While most of them were made to solve some current issues, good solutions were at the same time aiming for the future evolution.

IMMEDIATE SOLUTIONS VS. SYSTEM



System with
immediate solutions



Immediate solutions
with system

REMEDY 

If production is based on immediate solutions more than on system, it's time to revise the system and make some major updates. This new system should keep in mind original intentions of the immediate solutions and may keep some of them.

It's important to find a good time for changes.

As an example, let me show you the original artists workflow at Remedy and how we were making modular sets of 3d objects.

MODULAR SET PIPELINE

DCC
(3ds Max)

Property Editor
(Omni Tool)

World Editor



Our team consists mostly of Level Artists, who work on the edge of Level Art and Level Design. 3d-models are mostly outsourced.
One of the main concepts used – modular environment design.

MODULAR SET PIPELINE



Workflow:

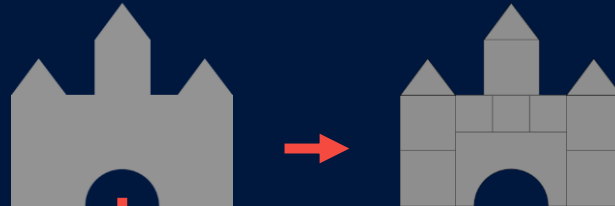
- Artists create a rough model of a building in 3dsMax
- Export to fbx
- Setup collisions in Omni Tool (our asset manager and property editor)
- Place in World Editor
- Test in Game

Iteration loop: change the model -> export -> if the model has new meshes, setup collision -> test in game

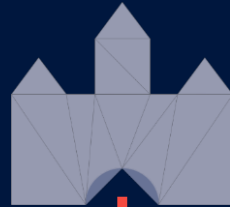
Weak point: setting up collisions. Patch: generate them based on the meshes names in Omni.

MODULAR SET PIPELINE

DCC
(3ds Max)



Property Editor
(Omni Tool)



World Editor



REMEDY 

When Level Artist is satisfied with the building, it's time to create modules for it.

- Make visual decomposition of the building
- Create needed modules while assembling the original building out of it. Doing that simultaneously is necessary as artists need to adjust modules and see, how they fit in the final building.

MODULAR SET PIPELINE



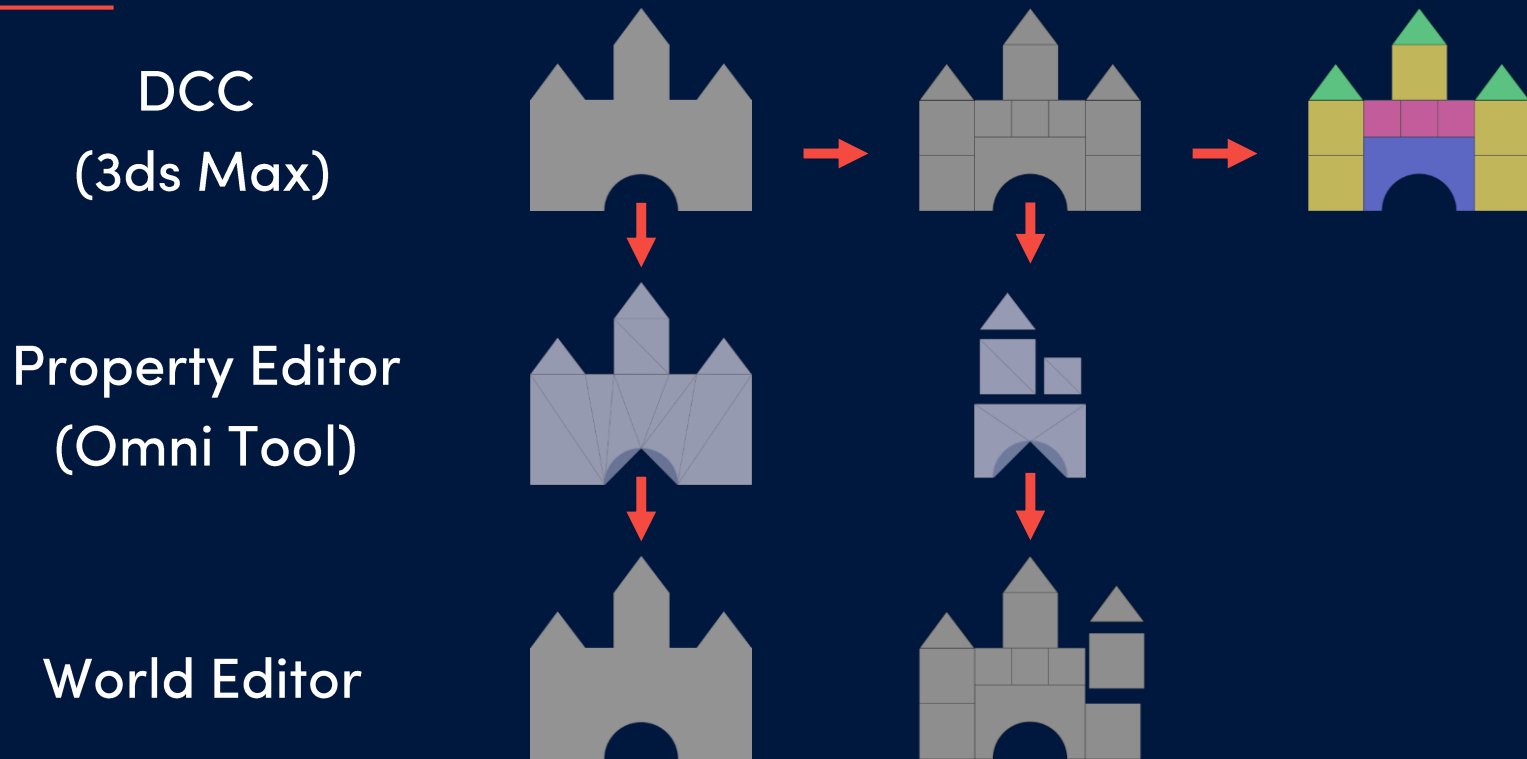
Now we have a module kit and assembled building in Max.

- Export modules one by one
- Setup collisions
- Reassemble building from modules in World Editor

Iteration loop: change module -> move instances in the building (to check if it still works) -> export -> reassemble in World Editor

Patch: ability to copy position in Max and paste in WED

MODULAR SET PIPELINE

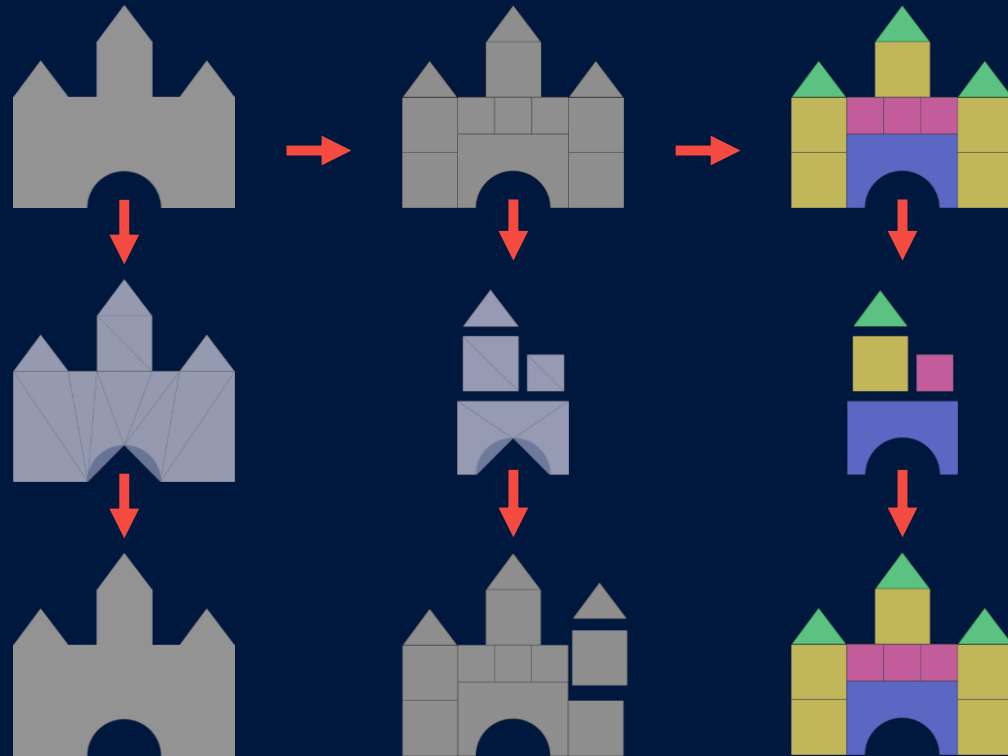


MODULAR SET PIPELINE

DCC
(3ds Max)

Property Editor
(Omni Tool)

World Editor



REMEDY 

- Create and assign needed materials
- Export meshes
- Create and setup materials in Omni Tool

Patch: "Create materials from FBX". Doesn't really work if you want to reuse materials across the meshes.

WHAT IS BAD?

- **Duplicate work**
 - Assembling modules
 - Creating materials
- **Redundant work**
 - Generating collision definitions
 - Assigning materials
- **A lot of context switching**

WHAT IS GOOD?

We've shipped games with it!

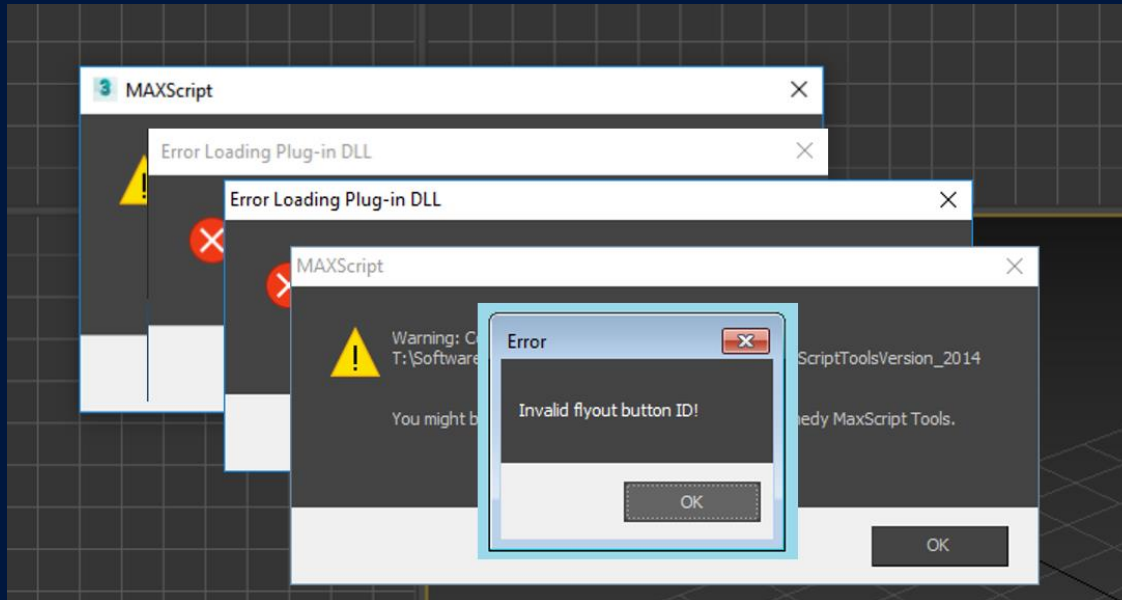


What is good? It works.

We have very talented artists, I was looking at how fast they work with that pipeline and thinking “how much faster can they be with the proper workflow”.

It means that people get used to any tools. And sometimes it creates bad practices.
Just look over the shoulder on how people work, and you will find a lot of unexpected things.

BAD PATTERNS



Once I saw artist opening 3ds Max file. Once file was opened, his thumb automatically started hitting the Space bar to close several appearing dialogs.

I asked if he had them every time and found that it's common issue of having unnecessary dialogs on startup. So artists just didn't report bugs and didn't believe the would be fixed.

This produces a vicious circle. User doesn't submit bugs -> developer doesn't fix bugs -> user doesn't submit bugs.

BAD PATTERNS



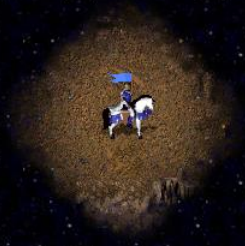
GOOD TIME FOR CHANGE



As you see, we needed to change something.
And it's important to find a good time for changes.

Our Tech Art team was lucky enough to find a good time for making steps forward. Traditionally, July is a month of vacation in Finland. So Remedy's office is not an exception: 70% of people are out – perfect time for RnD!

VISION FIRST



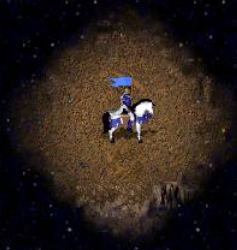
So let's take a look. We know:

- Where we are
- What issues we have

Before concrete actions we need to understand what do we want to achieve? Imagine an ideal result.

VISION FIRST

- “What do we want?”
- High-level requirements
- Can use User Stories as a tool:
 - ~~“Add support of importing 3dsMax scene in the World Editor”~~
 - “As an artist, I want to easily transfer assembled modular sets to the World Editor”



The answer should not consist of the technical solutions, but would rather have a list of “user stories”, very high-level definitions of requirements.

For example, compare :

- “Add support of importing 3dsMax scenes in the World Editor”
- “As an artist, I want to transfer assembled modular sets to the game faster”

Both call for solving the same issue, but the first restricts the technical solution while the second aims for improvement.

WHAT DO WE WANT?

- WYSIWYG
- Less context switching
- Predictable content
- Procedural workflows

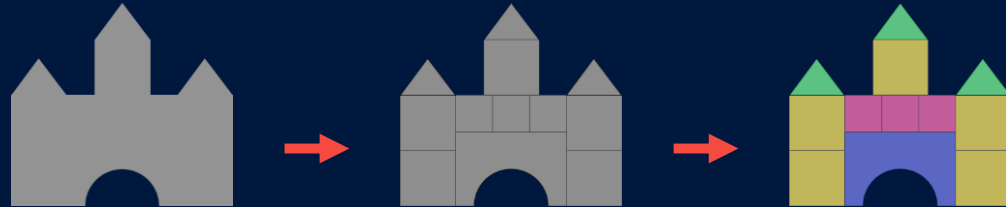


So what do we want to achieve with our Art pipeline?

- What you see in DCC is what you get in the engine – assembled modular sets in DCC should be easily transferred to the Engine and back. The same for materials
- Less context switching – more time artist spend in one application, less time he/she spend on alt+tab and less possibility of any of those to crash. If we need to set something up, do that in one application.
- Predictable content – even before launching the game engine, we want to be sure, that asset will be loaded correctly. Assets should respect naming conventions (or other meta-information) and follow other rules.
- Procedural workflows – something we didn't really have before, but will need in the new projects.

WHAT DO WE WANT?

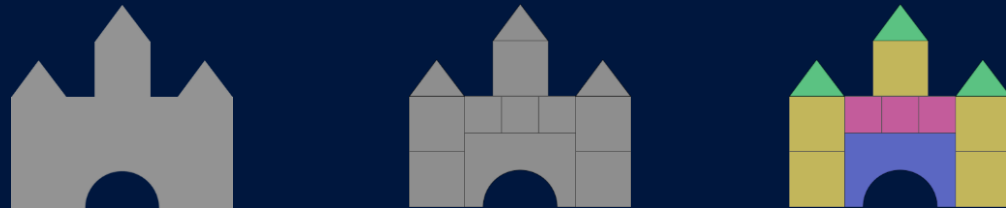
DCC



Property Editor
(Omni Tool)



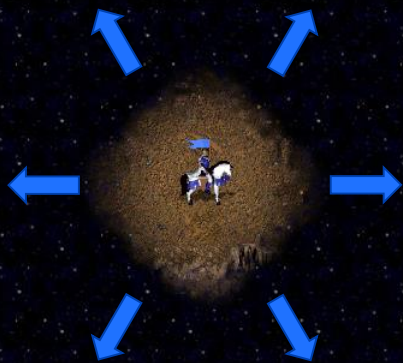
World Editor



REMEDY 

That's an ideal situation – artist may do the art in application that suits better for the current needs while having the same result in the other application.

SOLUTIONS



So we found the goal, what to do next?

There are a lot of solutions available in the industry (GDC provides a good portion of them). These solutions look tempting, but you rarely can just apply them out of the box. To find a proper one and adjust it, we must identify constraints the team operates under.

FINDING CONSTRAINTS



What technical limitations do we have?
What resources are available?
How much time do we have?

Those constraints don't give you an exact path to the goal but may help to find direction. They let you know the paths you cannot take or that may be difficult.

Addressing these questions help us define the direction that would ultimately guide the decision-making process later.

CONSTRAINTS AT REMEDY

- Proprietary game engine (Northlight)
- Tight tools team schedule
- Multiple projects
- Existing 3ds Max tools
- Experienced artists



Let's take a look at the constraints for Tech Artists in Remedy and try to define our path:

- Proprietary game engine (Northlight) - > No 3rd party solutions available -> Need to make a custom one
- Tight tools team schedule -> Long iteration loop -> Need to move iterations to the techart side
- Multiple projects -> Artists rotate between the projects -> Need to unify the workflow on the content creation side
- Existing 3ds Max tools -> Time (and money) spent on their development -> Hard to throw away -> Need to use them as a base
- Experienced artists -> Established workflows -> Existing culture that sometimes resists changes

DEFINING DIRECTION



Given all above we now can establish our main direction.
Decided: switching to Maya.
How come??

SWITCHING TO MAYA

- One DCC to rule them all
- Quick iteration loop
- Houdini Engine plugin
- Character Tech Team support



Let's see:

- One DCC = unified workflow, predictable content
- Quick iteration loop for tools for tech art (as we have more proficiency in Maya)
- Houdini Engine plugin to proof the concept before implementing it in the Northlight (saving tools team time)
- Also Character techart team uses Maya: library sharing and common launchers with character techart will help to optimize resources.

But we had a couple more constraints, right?

3DS MAX CONSTRAINT?

Tools used:

- Checkout 3ds Max file in P4
- Create “Remedy Root”
- Export selection



What about “3ds Max tools” constraint?

Can we just switch to Maya without regrets?

Fortunately, constraints may be challenged. We made a quick hook for tracking statistics and results gave us a clear answer:

Just 3 tools with basic functionality were regularly used.

We can mark the constraint as a “weak” and remember to bring some features from our Max tools.

Also I mentioned “experienced artists”. What about them?

TIME TO TALK

A man in profile, looking out over a city skyline at sunset. The sky is a deep red, and the city lights are visible in the distance. The man is wearing a dark jacket and a necklace.

I've got a bad feeling about this

We already made decision for ourselves, now we need to discuss it with the team. Get their reaction and understand if it is possible to proceed.

Changing 3d package is definitely not the news you want to present to artists.

Just imagine, we have a Senior Artist in the team who was working on the Max Payne. And I'm going to come to his desk and tell: "You know, you were working in the 3ds Max for the last 15 years, but now you are going switch to the Maya cause I have some ideas about Maya". Sounds a bit terrifying.

//Everything works, should we change?? Global changes are frightening.

PRESENTING IDEA

- Touching the ground
- Spreading idea
- Receiving feedback
- Getting people involved



The good thing is people at Remedy are very open to initiatives, so we talked to couple of artists and they were understanding. I would say, they were unexpectedly understanding.

Artists needed to have some time to think and discuss the idea.

In a few weeks after we've spoken with a few lead artists, people had started to ask me something like "I heard, we are moving to Maya? What to expect?" Speaking with people more about future abilities and technical details makes them involved in the process. They start thinking about their workflow in Maya, start giving you feedback on not yet existing tools.

Originally idea was ours, but now it's shared. That's when it became real, so there is no way back.

So turns out, the anxiety was mostly inside.

FIRST STEPS



Decision made, moving forward.
We don't know if every aspect is going to work, but we aim for it.
Not everything will be exactly as planned, but we always keep the picture in mind.

FIRST STEPS : POINT CLOUD

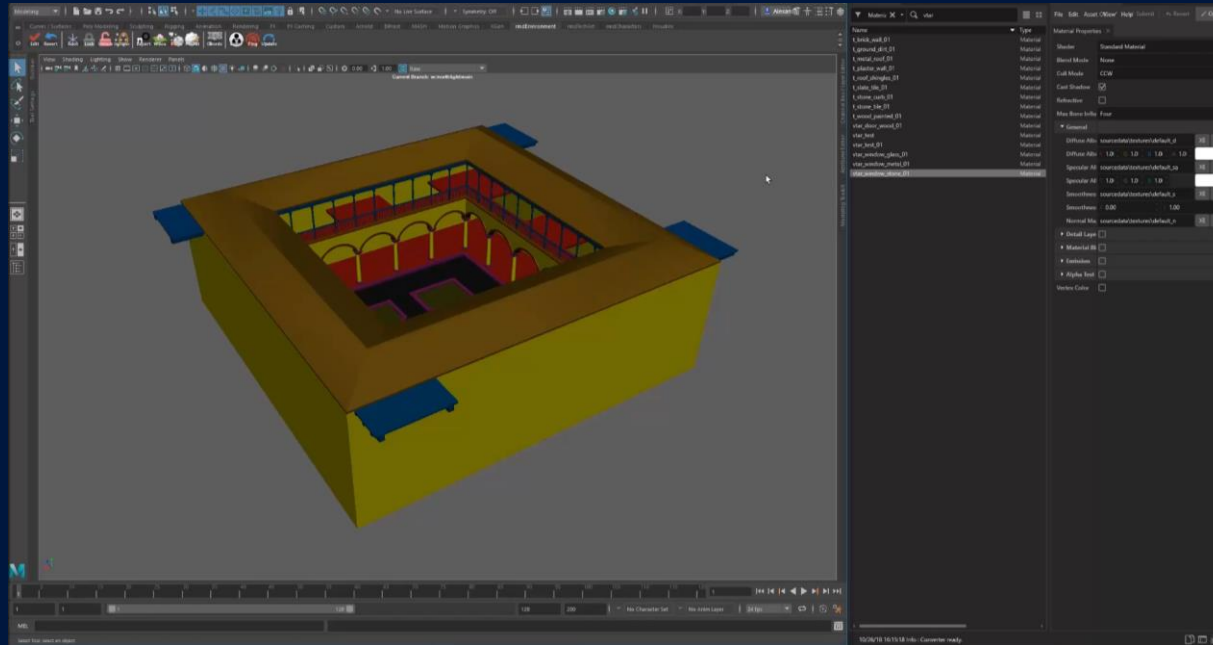


First things first. After making the basic tools for exporting and checking out scene in Perforce we started with making “proof of concept” prototypes.

To solve the issue of transferring assets, assembled from modules, we introduced Point Clouds. Point Cloud is a simple file format that for each instance stores its fbx path and transformation matrix. We can read and write this format in both Maya and game engine, which allows artists to make their modular layouts in Maya and see the result instantly in the World Editor.

As a side effect we got ability to easily export instance information from Houdini, which at the same moment allowed us to start testing procedural workflows.

FIRST STEPS : DRAG & DROP



One of the next things to solve was material assignment. We had to remove duplicate work and desynchronization between DCC and game engine.

We were definitely understanding, that artists needed to have all the proper materials assigned before they press the export button.

Without having a solid solution for that we made a prototype where we could drag'n'drop materials from the Omni Tool to Maya. Then on export we could use the path of the dropped material and automatically assign it in the engine. (Omni Tool, our standalone property editor, is on the right side of the image.)

Drag and drop is not a must have feature, but it explains the vision way better than any words.

TRANSITION: SPLASH SCREENS

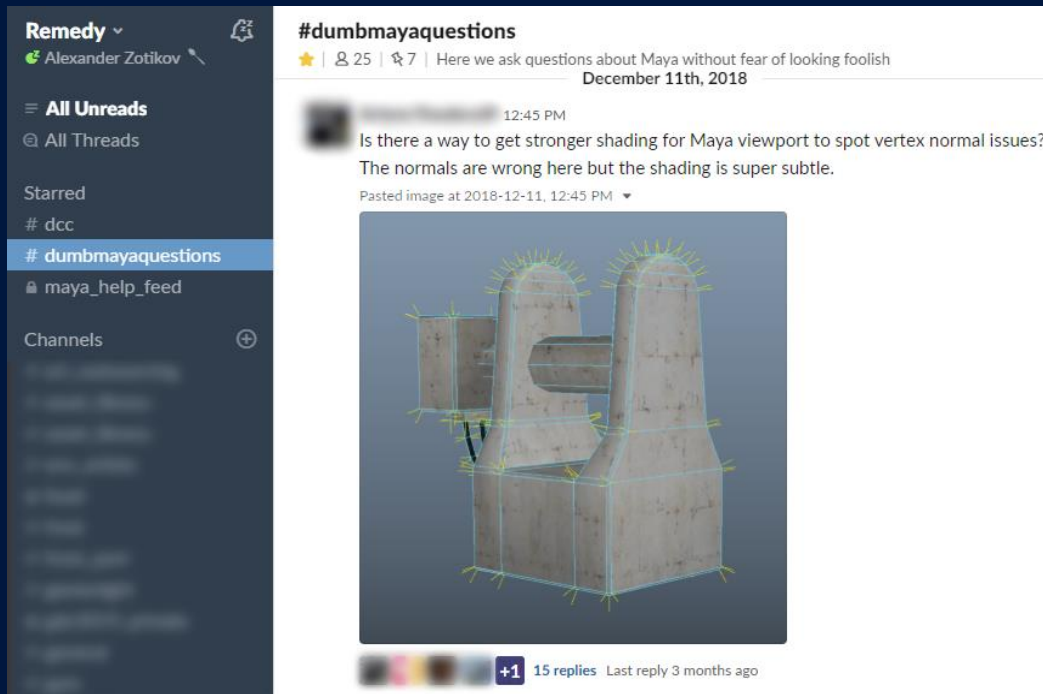


REMEDY

It was clear that we should've started smoothly and created a comfortable environment to remove discomfort from switching to the new DCC.

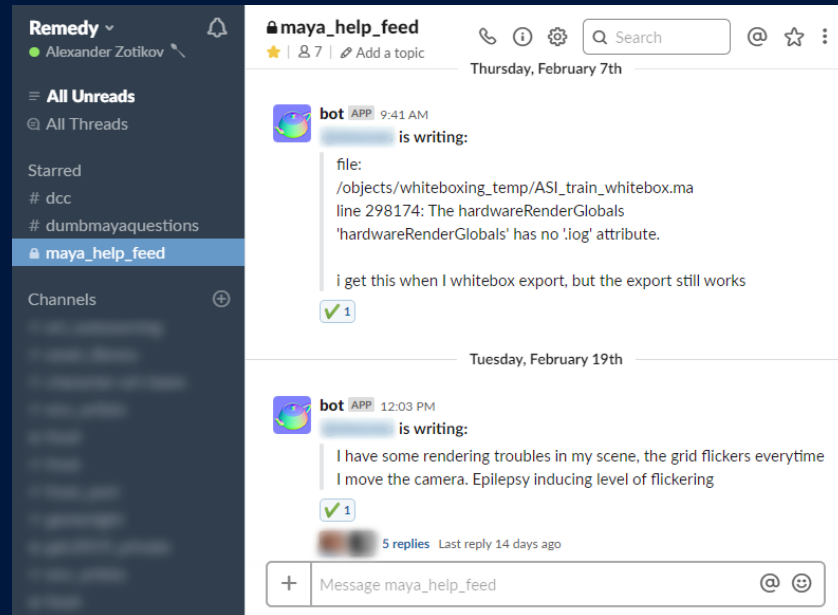
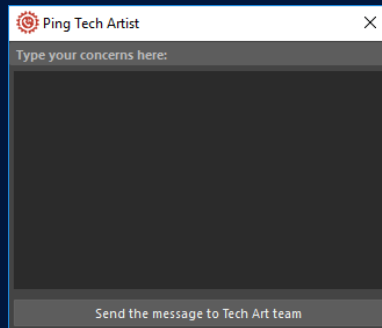
A small trick that can make Maya a bit friendly from the first launch is to change default splash screen to something more familiar. We've changed it to various images from the Remedy games.

TRANSITION : #DUMB MAYA QUESTIONS



Then we've created a #dumbmayaquestions Slack channel, where people without any hesitation can ask "how to open UV Editor" or "what do you mean, there are no smoothing groups in Maya?". Aside from just solving issues, it created a nice platform for sharing ideas about the new small tools.

TRANSITION : PING TA



Also, one of the first buttons we created was “Ping Technical Artist”. It sends user’s message via Slack to the Tech Art Team, so we can quickly react on the issues artists have. This simple and effective mechanism creates a “positive reinforcement” for the user. So, every time a user report an issue that we resolve, it encourages new to report the next issue.

Needless to say, that part of artists from the other project have already started working in Maya preparing for the future changes.

Total users per day (last 2 years)



EVEN TOOLS NEED A GOOD MARKETING

REMEDY 

It's clear, that even tools need a good marketing to support the tech.

DIRECTION ≠ PATH



No implementation should be set in stone. By doing that you may narrow down your possibility space for new solutions. Failing is a natural process during the evolution of ideas. So that's out false trail.

OUR FALSE TRAIL

Let's:

- Write data from Maya
- Do “everything else” in Maya
- Make a Maya Asset Browser



We had a naive idea that we would implement “everything artist need” on the Maya side without any extra support cause we didn’t want to bother the tools team.

One of the aspects was Maya Asset Browser to remove an extra context switching.

OUR FALSE TRAIL

Let's:

- Write data from Maya
- Do “everything else” in Maya
- Make a Maya Asset Browser

Questions:

- Code testing?
- Asset versions?
- Synchronizing Maya and Northlight code?



Tools team found out and asked reasonable questions:

- What about asset versions?
- What about code testing?
- What about synchronizing engine and Maya code?

Original idea of quick iteration loops faced the difficulties of real-life development.

OUR FALSE TRAIL

Let's:

- Write data from Maya
- Do “everything else” in Maya
- Make a Maya Asset Browser

Questions:

- Code testing?
- Asset versions?
- Synchronizing Maya and Northlight code?

Why do we need an Asset Browser in Maya
if there is one in Northlight?



The last question was: why do you need an Asset Browser in Maya if we have it in the game engine?
The fair answer turned out to be: because there is no thumbnails for assets and it's hard to search for something.

UPDATED PATH

- Drag & Drop as a main tool
- Use Northlight API
- Thumbnails in Omni Tool



So we decided to drop our original plan and to go with updated one:

- Using Omni Tool (Property Editor) as an Asset Browser for Maya (Drag n Drop now is a main concept, not just a prototype)
- Use engine API (well tested code) to run different actions for the assets
- Get thumbnails support on the engine side as a bonus (Yay!)

So talking is always a good idea.

TECHNICAL DETAILS: C#/PYTHON API

```
# Common Runtime Language
import clr

# Remy C#
from Remy.Assets import Fbx
from Remy.Properties.Serialization import OSPFileReader, OSPFileWriter

# Prepare objects
fbxScene = Fbx.FbxLoader().LoadScene(fbxFile).Result
metaObject = OSPFileReader(OSPService).Read(metaFile).Result

# Generate and write material bindings
Fbx.MaterialBindingGenerator.CreateMaterialBindings(fbxScene, metaObject)
OSPFileWriter().Write(metaObject, File.Get(filepath))
```



A bit about technical implementations on the new path.
Omni Tool (our Asset Browser) is written in C# and uses .NET framework.
This allow us to use API, provided by the tools programmers right from Python.

We just use “clr” package to load libraries in Python and then operate with data as with the native objects.
So we can easily read and modify the information about the assets and use well-tested functionality of Omni Tools.
This let us remove redundant steps from the pipeline as we setup most of the objects parameters right during the export.

TECHNICAL DETAILS: FILE TRANSLATOR

```
kPluginTranslatorTypeName = "RMDMaterialTranslator"

class MaterialTranslator(OpenMayaMPx.MPxFileTranslator):
    Extension = "material"

    def __init__(self):
        OpenMayaMPx.MPxFileTranslator.__init__(self)

    @classmethod
    def creator(cls):
        return OpenMayaMPx.asMPxPtr(cls())

    # Set filter in file selection dialogs
    def filter(self):
        return "*.{}".format(self.Extension)

    # Called when it's necessary to load the file of this type
    def reader(self, fileObject, optionString, accessMode):
        filePath = fileObject.fullName()
        materialFactory.Instance.createMaterial(filePath, filePath, materials.MayaMaterialBlinn)

    def initializePlugin(mobject):
        plugin = OpenMayaMPx.MFnPlugin(mobject)
        plugin.registerFileTranslator(kPluginTranslatorTypeName, None, MaterialTranslator.creator)

    def uninitializePlugin(mobject):
        plugin = OpenMayaMPx.MFnPlugin(mobject)
        plugin.deregisterFileTranslator(kPluginTranslatorTypeName)
```



File Translator is a mechanism in Maya to interpret files of different extensions. Once you write it for needed extension, you can import, export, reference files of any formats.

We wrote File Translators for our engine side assets: materials and pointclouds. Now we use drag'n'drop from Asset Manager and reference objects in Maya.

TECHNICAL DETAILS: HOT LOADING



File Translator allows us to easily reload references with Maya's build-in "reload". We run file watcher for each file in the background and reload references on request. So now, changing the texture of material in the Asset Manager will update it automatically in Maya. Also when you open your scene, you get the last updates of the referenced objects from Perforce

#REMIND_ME_LATER

- Check the system behind the patches
- Find your vision
- Leverage constraints
- Don't underestimate colleagues will for changes
- Change on a fly
- Talk and talk



REMEDY 

Making the first step doesn't mean achieving the result, but without it you cannot really move forward. Our Maya tools are still in development, we iterate on the workflow as well. But we clearly see now, why we made this uneasy move.

I created this small list of statements to remind myself from time to time to step back and look at the bigger picture instead of firefighting the issues via immediate solutions.

THANK YOU

We are hiring:
remedygames.com/careers

I am tweeting:
[@z_fighting](https://twitter.com/z_fighting)

If you are ready for challenges and interesting projects take a look at open positions at Remedy.
If you have any questions about the presentation or just want to chat, feel free to ping me on Twitter: [@z_fighting](https://twitter.com/z_fighting)

References:
To create the slides with the maps I used *Heroes Might & Magic 3* map editor provided with the original game from *New World Computing*